A

PROJECT REPORT

ON

FOOD DELIVERY DATA ANALYSIS

Identifying Factors that Influence Students' Marks and Overall Performance

OF

BACHELOR OF TECHNOLOGY

Computer Engineering

SUBMITTED BY

Pranav Shingne ISTU25020010257

Om More ISTU25020010251



DEPARTMENT OF COMPUTER ENGINEERING

G H RAISONI INTERNATIONAL SKILL TECH UNIVERSITY, PUNE

2026-27

CERTIFICATE

This is to certify that the project report entitled

FOOD DELIVERY DATA ANALYSIS

Submitted by

Pranav Shrinivas Shingne - Roll No: WTA167

Om More - Roll No: WTA166

is a Bonafide student of this institute and the work has been carried out by them under the supervision of **Prof. Zeba Vishwakarma** and it is approved for the partial fulfillment of the requirement of the G H Raisoni International Skill Tech University, Pune, for the award of the degree of **Bachelor of Technology in Computer Engineering** in the academic year 2026-27.

Prof. Zeba Vishwakarma
Subject Faculty

(Dr. S S Balwante)
H.O.D.

Date:

Place: Pune

ACKNOWLEDGMENT

It gives us great pleasure in presenting **Food Delivery Data Analysis** as my B.Tech. project. Words have never seemed as inadequate as now when I am endeavoring to express our gratitude at the culmination of our B. Tech. Project to all those who have made it possible. Even the best efforts are waste, without the proper guidance and advice of our project guide **Prof. Zeba Vishwakarma** for the consistent guidance, co-operation, inspiration, practical approach and constructive criticism, which provided us the much-needed impetus to work hard & also thanks **Dr. S S Balwante** Head of **Computer Engineering** Department for their continuous support & valuable suggestions.

We take this opportunity to thank **Dr. Sujit Deshpande Dean SOET** for her whole hearted support, motivation & valuable suggestions.

We would also like to thank **Prof. Zeba Vishwakarma** our Project Coordinators for their valuable support in providing us with the required information.

At the end, we would like to give special thanks to all staff members from **Computer Engineering Department** of G H Raisoni International Skill Tech University, Pune & our colleagues for their kind support & timely suggestions.

Pranav Shingne - Roll No: WTA167

Om More - Roll No: WTA166

ABSTRACT

Food delivery platforms generate large amounts of data related to customer orders, restaurant preferences, delivery times, payment methods, and spending patterns. Analysing this data can help identify popular cuisines, understand customer behaviour, and evaluate order and delivery trends. This project, **Food Delivery Data Analysis**, applies the concepts of Foundations of Data Science (FDS) to a food delivery dataset to extract meaningful insights through data preprocessing, exploratory data analysis (EDA), descriptive statistics, and data visualization.

The dataset, named *Swiggy_Data*, contains 108,384 records and 19 attributes related to food orders, customers, restaurants, cities, cuisines, payment methods, ratings, and delivery information. The dataset was processed using Python to examine data types, validate values, check for missing and duplicate records, and identify potential outliers using the Interquartile Range (IQR) method. Statistical measures such as mean, median, mode, variance, standard deviation, and correlation were used to understand the characteristics and relationships within the data. Various visualizations, including bar charts, histograms, pie charts, box plots, scatter plots, and correlation heatmaps, were used to present the findings effectively. The project was implemented using Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn.

The analysis examines order status, customer spending, cuisine preferences, restaurant performance, payment methods, customer ratings, and delivery-related patterns. It also uses statistical analysis and visual representations to identify trends, compare categories, and understand relationships between relevant variables. The results provide insights into customer ordering behaviour and food delivery operations, which can support better decision-making for restaurants and food delivery platforms.

This project demonstrates the practical application of data cleaning, data wrangling, descriptive statistics, exploratory data analysis, statistical analysis, and visualization techniques. It highlights how raw food delivery data can be transformed into meaningful information for understanding customer preferences and operational trends.

Keywords: Food Delivery Data Analysis, Foundations of Data Science, Data Preprocessing, Exploratory Data Analysis, Descriptive Statistics, Data Visualization, Statistical Analysis, Python, Pandas, NumPy.

TABLE OF CONTENTS

Sr. No.	Title of Chapter	Page No.
	CERTIFICATE	**2**
	ACKNOWLEDGMENT	**3**
	ABSTRACT	**4**
	TABLE OF CONTENTS	**5**
	LIST OF FIGURES	**7**
	LIST OF TABLES	**8**
	LIST OF ABBREVIATIONS	**9**
01	**Chapter 1: Introduction**	**10**
	1.1 Background of the Project	10
	1.2 Problem Statement	10
	1.3 Motivation	10
	1.4 Objectives	11
	1.5 Scope of the Project	12
02	**Chapter 2: Dataset Description**	**13**
	2.1 Dataset Source	13
	2.2 Dataset Overview	13
	2.3 Description of Attributes/Variables	13
	2.4 Data Types	15
	2.5 Dataset Size – Rows and Columns	16
03	**Chapter 3: Data Preprocessing and Exploratory Analysis**	**17**
	3.1 Data Loading	17
	3.2 Data Cleaning	17
	3.2.1 Handling Missing Values	17
	3.2.2 Handling Duplicate Values	18
	3.2.3 Data Type Conversion	18
	3.2.4 Outlier Detection and Handling	19
	3.3 Exploratory Data Analysis	20
	3.3.1 Descriptive Statistics	20
	3.3.2 Measures of Central Tendency	20
	3.3.3 Measures of Dispersion	21
	3.3.4 Frequency Analysis	21
	3.3.5 Data Aggregation and Grouping	23
	3.3.6 Correlation Analysis	24
	3.4 Data Visualization	25
	3.4.1 (Count Plot) Order Status Distribution	25

Sr. No.	Title of Chapter	Page No.
	3.4.2 (Line Chart) Monthly Order Trends	27
	3.4.3 (Horizontal Bar Chart) Top 10 Popular Cuisines	28
	3.4.4 (Bar Chart) Order Distribution by Hour	29
	3.4.5 (Histogram with KDE) Order Amount Distribution	31
	3.4.6 (Line Chart) Monthly Spending Trends	32
	3.5 Statistical Analysis	34
	3.5.1 Statistical Methods Used	34
	3.5.2 Probability / Distribution Analysis	34
	3.5.3 Correlation	35
	3.5.4 Simple Linear Regression	35
	3.5.5 Interpretation of Statistical Results	35
04	**Chapter 4: Results and Findings**	**36**
	4.1 Key Observations	36
	4.2 Important Patterns Identified	36
	4.3 Summary of Results	37
05	**Chapter 5: Conclusion and Future Scope**	**38**
	5.1 Conclusion	38
	5.2 Limitations	38
	5.3 Future Scope	39
	REFERENCES	**40**
	APPENDIX – PYTHON CODE	**42**

LIST OF FIGURES

FIGURE NO.	ILLUSTRATION	PAGE NO.
3.1	Distribution of Order Status	18
3.2	Monthly Order Trends	19
3.3	Top 10 Popular Cuisines by Order Volume	19
3.4	Order Distribution by Recorded Delivery Hour	20
3.5	Distribution of Food Order Amounts	20
3.6	Monthly Food Order Spending Trends	21

LIST OF TABLES

TABLE NO.	ILLUSTRATION	PAGE NO.
2.1	Description of Dataset Attributes	12
2.2	Summary of Dataset Data Types	13
3.1	Missing Values and Data Cleaning Summary	14
3.2	Descriptive Statistics of Numerical Variables	15
3.3	Measures of Central Tendency for Numerical Variables	15
3.4	Measures of Dispersion and Skewness	16
3.5	Frequency Distribution of Order Status and Payment Methods	16
3.6	Average Order Amount by Cuisine or Restaurant	17
3.7	Average Order Amount by Order Status and Customer Groups	17
3.8	Correlation of Numerical Variables	18
3.9	Summary of Statistical Analysis	23
4.1	Summary of Key Findings from Food Delivery Data Analysis	24

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
FDS	Foundations of Data Science
EDA	Exploratory Data Analysis
IQR	Interquartile Range
CSV	Comma-Separated Values
SD / Std	Standard Deviation
ANOVA	Analysis of Variance
R^2	Coefficient of Determination
r	Pearson Correlation Coefficient
ID	Identifier

Chapter 1: Introduction

1.1 Background of the Project

Food delivery platforms have become an important part of the modern food service industry. These platforms generate large amounts of data related to customer orders, restaurant details, cuisines, payment methods, customer ratings, delivery information, and spending patterns. Analysing this data can help understand customer preferences, identify popular food categories, and examine trends in food delivery operations.

Data Science provides systematic methods for converting raw data into meaningful information. The techniques covered in the Foundations of Data Science (FDS) course, such as data preprocessing, exploratory data analysis (EDA), descriptive statistics, statistical analysis, and data visualization, can be applied to study food delivery data effectively.

This project, Food Delivery Data Analysis, uses a dataset containing food order and customer-related information to explore ordering patterns, popular cuisines, customer spending, payment preferences, restaurant performance, and delivery-related trends. Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn are used to clean, analyse, summarize, and visualize the dataset. The project demonstrates how data analysis techniques can transform raw food delivery records into useful insights for understanding customer behaviour and operational patterns.

1.2 Problem Statement

To analyse food delivery data in order to understand customer ordering patterns, popular cuisines, delivery-related trends, payment preferences, customer ratings, and spending behaviour. The project applies data preprocessing, exploratory data analysis (EDA), descriptive statistics, statistical analysis, and visualization techniques from the Foundations of Data Science course to identify meaningful patterns and relationships in the dataset.

1.3 Motivation

- Food delivery platforms generate large amounts of data, but systematic analysis is required to convert these records into useful insights.
- Understanding popular cuisines and ordering patterns can help identify customer preferences.
- Analysing order amounts and payment methods provides insights into customer spending

and payment behaviour.

- Examining order status, ratings, and delivery-related information can help identify operational patterns and areas that may require improvement.
- The project provides practical experience in applying FDS concepts, including data cleaning, data wrangling, descriptive statistics, exploratory data analysis, and visualization, to a real-world-style dataset.

1.4 Objectives

- To examine the structure, attributes, and data types of the food delivery dataset.
- To identify and handle missing values, duplicate records, inconsistent data, and invalid values where applicable.
- To analyse the distribution of orders across cities, restaurants, and cuisines.
- To study order status and compare the number of delivered and cancelled orders.
- To analyse customer spending patterns using statistical measures such as mean, median, mode, variance, and standard deviation.
- To examine the distribution of customer ratings and understand restaurant-related rating patterns.
- To analyse payment method preferences among customers.
- To investigate delivery-related patterns using the available delivery information.
- To identify correlations and relationships between relevant numerical variables.
- To present the findings using appropriate charts, tables, and statistical summaries.

1.5 Scope of the Project

The scope of this project is limited to the exploratory and statistical analysis of the *Swiggy_Data* food delivery dataset, which contains 108,384 records and 19 attributes related to orders, customers, restaurants, and delivery information.

The analysis covers the following areas:

- **Order analysis:** Distribution of orders, order status, and ordering patterns.
- **Cuisine and restaurant analysis:** Popular cuisines, restaurant activity, and related ratings.
- **Customer spending analysis:** Distribution of order amounts and measures of central tendency and dispersion.
- **Payment analysis:** Comparison of payment methods used for food orders.
- **Rating analysis:** Examination of customer ratings and their distribution.
- **Delivery-related analysis:** Exploration of available delivery-time information and related patterns.
- **Statistical analysis:** Use of descriptive statistics, correlation, and other applicable analytical techniques.
- **Data visualization:** Presentation of findings through bar charts, histograms, pie charts, box plots, scatter plots, and correlation heatmaps, where appropriate.

The project focuses on data preprocessing, exploratory data analysis, descriptive statistics, and visualization using Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn. Advanced predictive machine-learning models and real-time food delivery system development are outside the current scope and may be considered under Future Scope.

Chapter 2: Dataset Description

2.1 Dataset Source

The dataset used in this project is named *Swiggy_Data* and contains food delivery order information related to customers, restaurants, cuisines, payment methods, ratings, and delivery details. It is used to study food ordering patterns, customer spending behaviour, popular cuisines, order status, and other relevant trends in food delivery operations.

The dataset contains 108,384 records and 19 attributes. It is analysed using Python and the data analysis libraries Pandas, NumPy, Matplotlib, and Seaborn. The dataset is loaded from a CSV file named Swiggy_Data.csv. For reproducibility, the file can be placed in the specified Google Drive folder or uploaded directly in Google Colab.

The dataset is used for academic purposes to demonstrate the practical application of Foundations of Data Science (FDS) concepts, including data preprocessing, data validation, exploratory data analysis (EDA), descriptive statistics, and data visualization.

2.2 Dataset Overview

The dataset contains **108,384 rows and 19 columns**. Each record represents a food delivery order and includes information about the order, customer, restaurant, payment method, rating, and delivery details. These attributes support the analysis of customer preferences, order amounts, restaurant activity, payment patterns, and order outcomes.

Unlike a student performance dataset, this project does not use a single academic target variable. Instead, it examines different attributes according to the objective of each analysis. For example, total_amount is used to study customer spending, order_status to examine order outcomes, cuisine to identify cuisine preferences, and rating to analyse customer ratings.

2.3 Description of Attributes/Variables

Table 2.1: Description of dataset attributes

Sr.	Attribute	Description	Type
1	order_id	Unique identifier for each order	Identifier
2	user_id	Identifier of the customer who placed the order	Identifier
3	restaurant_id	Unique identifier of the restaurant	Identifier
4	order_date	Date on which the order was placed	Date
5	delivery_time	Delivery-related time value recorded in the	Time / recorded value

		dataset	
6	order_status	Status of the order, such as Delivered or Cancelled	Categorical
7	payment_method	Payment method used for the order	Categorical
8	total_amount	Total monetary amount of the order	Numerical
9	restaurant_name	Name of the restaurant associated with the order	Categorical
10	city	City associated with the order or restaurant	Categorical
11	cuisine	Cuisine category offered by the restaurant	Categorical
12	rating	Customer rating recorded for the order or restaurant	Numerical
13	is_cloud_kitchen	Indicates whether the restaurant operates as a cloud kitchen	Binary / categorical
14	user_name	Name associated with the customer record	Text
15	age	Age of the customer	Numerical
16	gender	Gender category recorded for the customer	Categorical
17	marital_status	Marital status recorded for the customer	Categorical
18	occupation	Occupation category of the customer	Categorical

2.4 Data Types

The dataset contains a combination of numerical, categorical, identifier, date, time, and text attributes. Numerical variables such as total_amount, rating, and age can be examined using statistical measures. Categorical variables such as order_status, payment_method, cuisine, city, gender, marital_status, and occupation are useful for frequency analysis and group comparisons.

Identifier fields, including order_id, user_id, and restaurant_id, are primarily used to identify records and should not normally be treated as numerical measurements. The order_date field is used for date-based analysis after conversion to a suitable date-time format.

Table 2.2: Summary of data types

Type	Count	Attributes
dentifier	`order_id`, `user_id`, `restaurant_id`	Identifying orders, customers, and restaurants
Date	`order_date`	Analysing order dates and time-based patterns
Time / recorded value	`delivery_time`	Examining the recorded delivery-related time information
Numerical	`total_amount`, `rating`, `age`	Statistical summaries and numerical comparisons
Binary / categorical	`is_cloud_kitchen`	Comparing cloud kitchens with other restaurant types
Categorical / text	`order_status`, `payment_method`, `restaurant_name`, `city`, `cuisine`, `user_name`, `gender`, `marital_status`, `occupation`	Frequency distributions, group comparisons, and category analysis

2.5 Dataset Size – Rows and Columns

The dataset contains **108,384 rows and 19 columns**, representing food delivery orders and their associated customer, restaurant, payment, rating, and delivery information.

During preprocessing, the dataset is inspected for missing values, duplicate records, invalid values, inconsistent data types, and potential outliers. Any changes to the number of rows or columns depend on the cleaning operations actually performed. The final dataset dimensions should therefore be recorded after executing the preprocessing steps.

The cleaned dataset is subsequently used for exploratory data analysis, descriptive statistics, statistical analysis, and visualization. These techniques help summarize the data and identify patterns related to order status, cuisine preferences, customer spending, payment methods, customer ratings, and delivery-related information.

Chapter 3: Data Preprocessing and Exploratory Analysis

3.1 Data Loading

The food delivery dataset, named *Swiggy_Data*, was loaded into a Pandas DataFrame for analysis. The read_csv() function was used to import the CSV file. The head(), info(), shape, and describe() functions were used to inspect the initial records, column names, data types, dataset dimensions, and descriptive statistics.

The dataset contains 108,384 records. The exact number of columns and their data types were verified during the loading and inspection process.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv(
    "/content/drive/MyDrive/Food_Delivery_Data_Analysis/Swiggy_Data.csv"
)

print("Dataset shape:", df.shape)
display(df.head())
df.info()
display(df.describe(include="all").T)
```

3.2 Data Cleaning

Data cleaning is an important preprocessing step because missing values, duplicate records, inconsistent formats, and invalid values can affect the accuracy of the analysis. The dataset was examined to identify potential data quality issues before conducting exploratory data analysis.

3.2.1 Handling Missing Values

The isnull().sum() function was used to identify missing values in each column. The treatment of missing values depends on the data type and the purpose of the column. Numerical values may be filled using the median or mean where appropriate, while categorical values may be filled using the mode or a separate category such as “Unknown”.

Table 3.1: Missing values and treatment applied

Column	Missing values	Treatment
Numerical columns	0	Apply a suitable method if required
Categorical columns	0	Use mode or “Unknown” where appropriate
Date/time columns	0	Correct formats or investigate invalid values

```
missing_values = df.isnull().sum()

missing_summary = pd.DataFrame({
    "Missing Values": missing_values,
    "Percentage": (missing_values / len(df) * 100).round(2)
})

display(missing_summary)
```

3.2.2 Handling Duplicate Values

Duplicate records were checked using duplicated().sum(). Duplicate rows can distort frequency counts, average order amounts, and other statistical summaries. Exact duplicate rows may be removed when confirmed to be redundant.

```
print("Duplicate rows:", df.duplicated().sum())

# Remove exact duplicate rows only if this is part of the actual cleaning process
df = df.drop_duplicates()

print("Dataset shape after duplicate-row removal:", df.shape)
```

3.2.3 Data Type Conversion

The data types of the columns were examined using df.info() and df.dtypes. Date values were checked to ensure that order_date was interpreted correctly. Numerical columns such as total_amount, rating, and age were examined for appropriate numerical types, while categorical columns such as city, cuisine, payment_method, and order_status were reviewed for consistent values.

```
print(df.dtypes)

# Convert order_date only if required by the existing dataset
df["order_date"] = pd.to_datetime(
    df["order_date"],
    errors="coerce"
)

print("Invalid or missing dates:", df["order_date"].isna().sum())
```

The delivery_time column requires special attention. If its values represent clock times such as 12:54:00, they should not automatically be treated as delivery durations. A delivery-duration analysis requires a duration field or both order and delivery timestamps.

Derived columns, such as order month or delivery hour, may be created when required for the analysis. These columns should be documented separately from the original dataset attributes

3.2.4 Outlier Detection and Handling

The Interquartile Range (IQR) method can be used to identify unusually low or high numerical values. The IQR is calculated as the difference between the third quartile (Q3) and the first quartile (Q1).

[
IQR = Q3-Q1
]

The lower and upper fences are calculated as:

[
\text{Lower Fence}=Q1-1.5(IQR)
]

[
\text{Upper Fence}=Q3+1.5(IQR)
]

In this project, IQR analysis can be applied to total_amount and age. Values outside the calculated fences are potential outliers, not automatically errors. For example, an unusually expensive order may be valid, so the original records should be examined before deciding whether to retain, correct, cap, or remove any value.

```
target_cols = ["total_amount", "age"]

for col in target_cols:
    values = pd.to_numeric(df[col], errors="coerce").dropna()

    q1 = values.quantile(0.25)
    q3 = values.quantile(0.75)
    iqr = q3 - q1

    lower_fence = q1 - 1.5 * iqr
    upper_fence = q3 + 1.5 * iqr

    outliers = values[
        (values < lower_fence) |
        (values > upper_fence)
    ]

    print(f"\nColumn: {col}")
    print("Q1:", q1)
    print("Q3:", q3)
    print("IQR:", iqr)
```

```
    print("Lower fence:", lower_fence)
    print("Upper fence:", upper_fence)
    print("Number of potential outliers:", len(outliers))
```

3.3 Exploratory Data Analysis

Exploratory Data Analysis (EDA) is used to understand the main characteristics of the dataset through descriptive statistics, frequency distributions, grouping, and visualization. The food delivery analysis examines ordering patterns, popular cuisines, customer spending, order status, payment methods, ratings, and available delivery-time information.

3.3.1 Descriptive Statistics

The describe() function was used to summarize numerical variables. For this dataset, relevant numerical attributes include total_amount, rating, and age. Additional numerical variables may be included if they are present in the dataset or created during preprocessing.

Table 3.2: Descriptive statistics of numerical variables

Variable	Count	Mean	Std	Min	25%	50%	75%	Max
Total Amount	108384.0	693.97928	440.412559	30.54	366.2075	608.025	932.355	3998.53
Rating	108384.0	4.051888	0.597977	2.5	3.5	4.0	4.5	5.0
Age	108384.0	41.199827	13.53191	18.0	30.0	41.0	53.0	64.0

```
numerical_cols = ["total_amount", "rating", "age"]

display(
    df[numerical_cols]
    .describe()
    .T
    .round(2)
)
```

3.3.2 Measures of Central Tendency

Mean, median, and mode are measures of central tendency used to describe the typical value of a variable. In this project, these measures can be calculated for order amounts, customer ages, and ratings.

Table 3.3: Measures of central tendency

Variable	Mean	Median	Mode
Total Amount	693.97928	608.02	154.12
Rating	4.051888	4.00	4.50

Age	41.199827	41.00	43.00

```
for col in numerical_cols:
    values = pd.to_numeric(df[col], errors="coerce").dropna()

    print(f"\n{col}")
    print("Mean:", round(values.mean(), 2))
    print("Median:", round(values.median(), 2))
    print("Mode:", values.mode().tolist())
```

3.3.3 Measures of Dispersion

Measures of dispersion describe how widely observations vary. Variance, standard deviation, range, and IQR can be calculated for the numerical attributes.

Table 3.4: Measures of dispersion and skewness

Variable	Variance	Std. Dev.	Range	IQR
Total Amount	193963.22	440.41	3998.53	566.15
Rating	0.36	0.60	5.00	-
Age	183.11	13.53	64.00	23.00

```
for col in numerical_cols:
    values = pd.to_numeric(df[col], errors="coerce").dropna()

    print(f"\n{col}")
    print("Variance:", round(values.var(), 2))
    print("Standard deviation:", round(values.std(), 2))
    print("Range:", round(values.max() - values.min(), 2))
    print("IQR:", round(
        values.quantile(0.75) - values.quantile(0.25), 2
    ))
```

These measures help determine whether customer spending and ratings are relatively consistent or vary substantially across records.

3.3.4 Frequency Analysis

Frequency analysis counts how often each category appears. The value_counts() function can be used to examine order status, payment method, city, cuisine, and cloud-kitchen status.

Table 3.5: Frequency Distribution of Order Status and Payment Methods

Order Status Distribution

Category	Frequency	Percentage
Delivered	103,019	95.05%
Cancelled	5,365	4.95%
Total	108,384	100.00%

Payment Method Distribution

Category	Frequency	Percentage
UPI	43,409	40.05%
Credit Card	27,197	25.09%
Debit Card	16,216	14.96%
Wallet	10,854	10.01%
Cash on Delivery (COD)	10,708	9.88%
Total	108,384	100.00%

```
for col in ["order_status", "payment_method"]:
    counts = df[col].value_counts(dropna=False)

    summary = pd.DataFrame({
        "Frequency": counts,
        "Percentage": (counts / len(df) * 100).round(2)
    })

    print(f"\n{col}")
    display(summary)
```

Frequency analysis shows which order statuses and payment methods are most common. Similar analysis can identify the cities and cuisines with the largest number of recorded orders. The results should be interpreted as counts in the dataset, not necessarily as market-wide popularity.

3.3.5 Data Aggregation and Grouping

The groupby() function was used to compare order counts and average order amounts across different categories. This analysis can help identify differences between cities, cuisines, restaurants, payment methods, and order statuses.

Table 3.6: Average Order Amount by Cuisine

Cuisine	Number of Orders	Average Order Amount
Seafood	6362	705.39
Beverages	6319	698.28
Continental	6316	682.79
Italian	6238	705.60
Desserts	5993	705.70
Thai	5963	679.37
Cafe	5766	688.10
Street Food	5545	696.16
Chinese	5527	707.00
Healthy Food	5380	677.97

```
cuisine_analysis = df.groupby('cuisine').agg(
    Number_of_Orders=('order_id', 'count'),
    Average_Order_Amount=('total_amount', 'mean')
).sort_values('Number_of_Orders', ascending=False)


cuisine_analysis['Average_Order_Amount'] = (
    cuisine_analysis['Average_Order_Amount'].round(2)
)
```

Table 3.7: Order Analysis by City and Order Status

City	Status	Number of Orders	Average Order Amount
Mumbai	**Delivered**	22809	693.02
Delhi	**Delivered**	18524	691.53
Bangalore	**Delivered**	15303	699.95

Hyderabad	Delivered	14193	693.04
Chennai	Delivered	9405	679.54
Kolkata	Delivered	7841	684.81
Pune	Delivered	6642	707.38
Jaipur	Delivered	3171	700.95
Ahmedabad	Delivered	2594	692.40
Lucknow	Delivered	2537	704.52
Mumbai	Cancelled	1195	690.00
Delhi	Cancelled	1007	713.18
Bangalore	Cancelled	787	701.45
Hyderabad	Cancelled	730	686.82
Chennai	Cancelled	468	751.88
Kolkata	Cancelled	395	700.80
Pune	Cancelled	364	712.67
Jaipur	Cancelled	171	729.34
Ahmedabad	Cancelled	125	737.90
Lucknow	Cancelled	123	693.64

```
display(cuisine_analysis.head(10))

city_status_analysis = df.groupby(['city', 'order_status']).agg(
    Number_of_Orders=('order_id', 'count'),
    Average_Order_Amount=('total_amount', 'mean')
).round(2).sort_values('Number_of_Orders', ascending=False)

display(city_status_analysis)
```

These summaries help compare ordering activity and average spending across categories. If an order ID can occur in multiple rows, the count should be checked against the dataset structure before being described as a count of unique orders.

3.3.6 Correlation Analysis

Correlation analysis measures the strength and direction of the linear relationship between numerical variables. Pearson's correlation coefficient ranges from −1 to +1. A positive value indicates that two variables tend to increase together, while a negative value indicates that one tends to decrease as the other increases.

For this dataset, correlation can be examined between suitable numerical attributes such as total_amount, rating, and age. Categorical fields should not be included as if they were continuous numerical measurements.

Table 3.8: Correlation of each factor with Final_Marks

Variable Pair	Total Amount	Rating	Age
total_amount	1.00000	1.00000	1.00000
rating	0.11121	0.11121	0.11121
age	-0.00563	-0.00563	-0.00563

```
display(city_status_analysis)

correlation_matrix = df[
    ['total_amount', 'rating', 'age']
].corr(method='pearson')

print(correlation_matrix.round(5))
```

Correlation values describe association rather than causation. For example, a correlation between order amount and rating would not by itself establish that spending more causes a higher rating.

3.4 Data Visualization

Data visualization presents patterns in a graphical form and makes comparisons easier to interpret. Matplotlib and Seaborn can be used to generate charts for order distribution, spending, cuisine popularity, ratings, and relationships between numerical variables.

3.4.1 (Count Plot) Order Status Distribution

A count plot displays the number of delivered and cancelled food orders in the dataset. It helps evaluate order completion patterns and identify the proportion of unsuccessful transactions. This analysis provides insight into the overall status of food orders.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 5))
sns.set_theme(style="whitegrid")

sns.countplot(
```

```
    data=df,
    x='order_status',
    order=df['order_status'].value_counts().index,
    hue='order_status',
    palette='Set2',
    legend=False
)

plt.title("Distribution of Order Status", fontsize=14, pad=15)
plt.xlabel("Order Status", fontsize=12)
plt.ylabel("Number of Orders", fontsize=12)
plt.tight_layout()
plt.show()
```

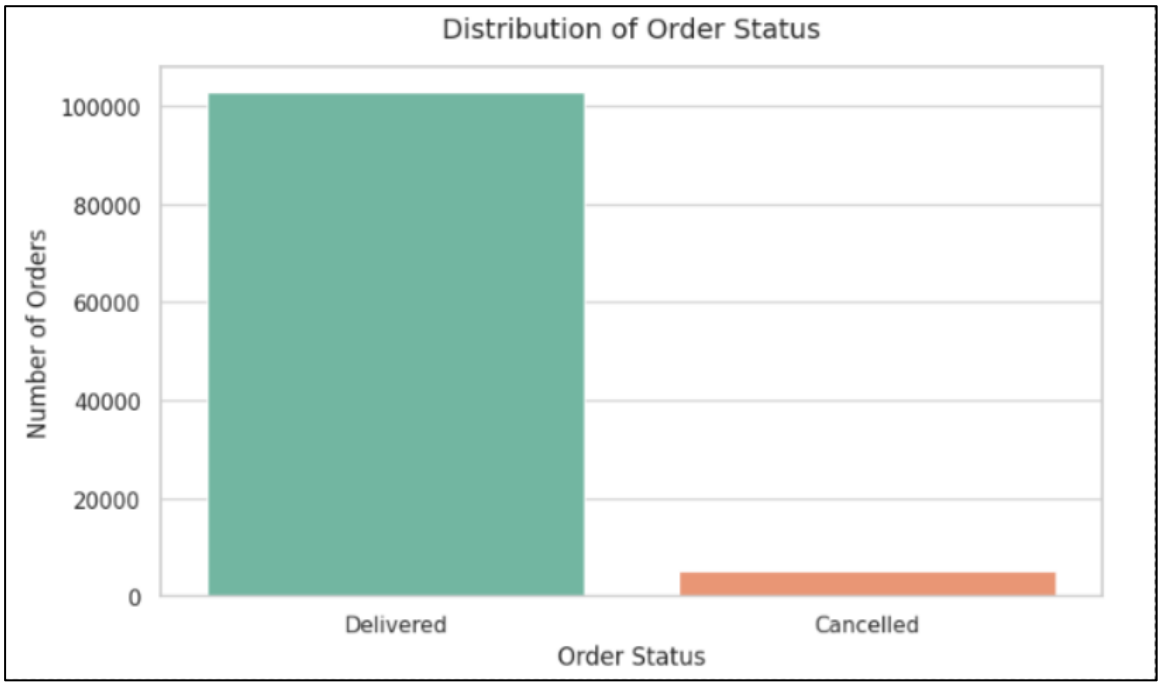


Figure 3.1: Distribution of Orders Status

3.4.2 (Line Chart) Monthly Order Trends

A line chart illustrates changes in the number of food orders across months. It helps identify monthly ordering patterns, increases or decreases in order volume, and changes in customer activity over time.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

df['order_date'] = pd.to_datetime(df['order_date'], errors='coerce')

monthly_orders = (
    df.dropna(subset=['order_date'])
      .set_index('order_date')
      .resample('ME')
      .size()
      .reset_index(name='Order Count')
)

monthly_orders['Month'] = monthly_orders['order_date'].dt.strftime('%b %Y')

plt.figure(figsize=(12, 6))

sns.lineplot(
    data=monthly_orders,
    x='Month',
    y='Order Count',
    marker='o',
    color='darkorange',
    linewidth=2.5
)
```

```
plt.title("Monthly Food Order Trends", fontsize=14, pad=15)
plt.xlabel("Month", fontsize=12)
plt.ylabel("Number of Orders", fontsize=12)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

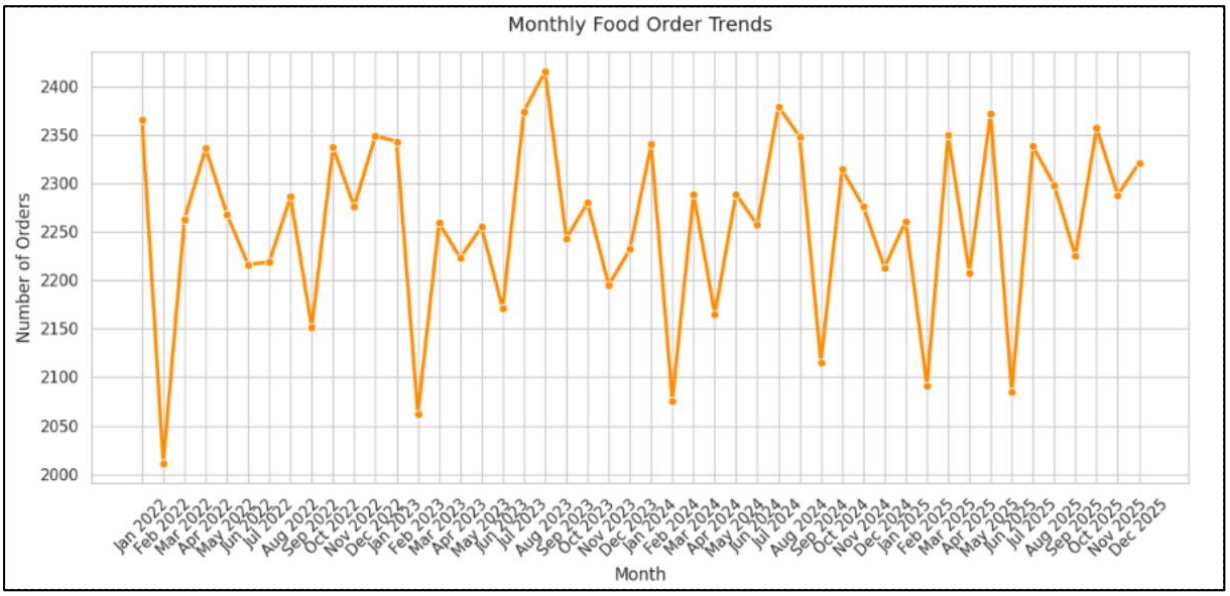


Figure 3.2: Monthly Order Trends

3.4.3 (Horizontal Bar Chart) Top 10 Popular Cuisines

A horizontal bar chart displays the ten most popular cuisines based on the number of food orders. It helps identify customer food preferences and compare the popularity of different cuisines in the dataset.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10, 6))
sns.set_theme(style="whitegrid")

top_cuisines = df['cuisine'].value_counts().head(10).reset_index()
top_cuisines.columns = ['Cuisine', 'Order Count']

sns.barplot(
    data=top_cuisines,
    x='Order Count',
    y='Cuisine',
    hue='Cuisine',
    palette='viridis',
```

```
    legend=False
)

plt.title("Top 10 Popular Cuisines by Order Volume", fontsize=14, pad=15)
plt.xlabel("Number of Orders", fontsize=12)
plt.ylabel("Cuisine", fontsize=12)
plt.tight_layout()
plt.show()
```

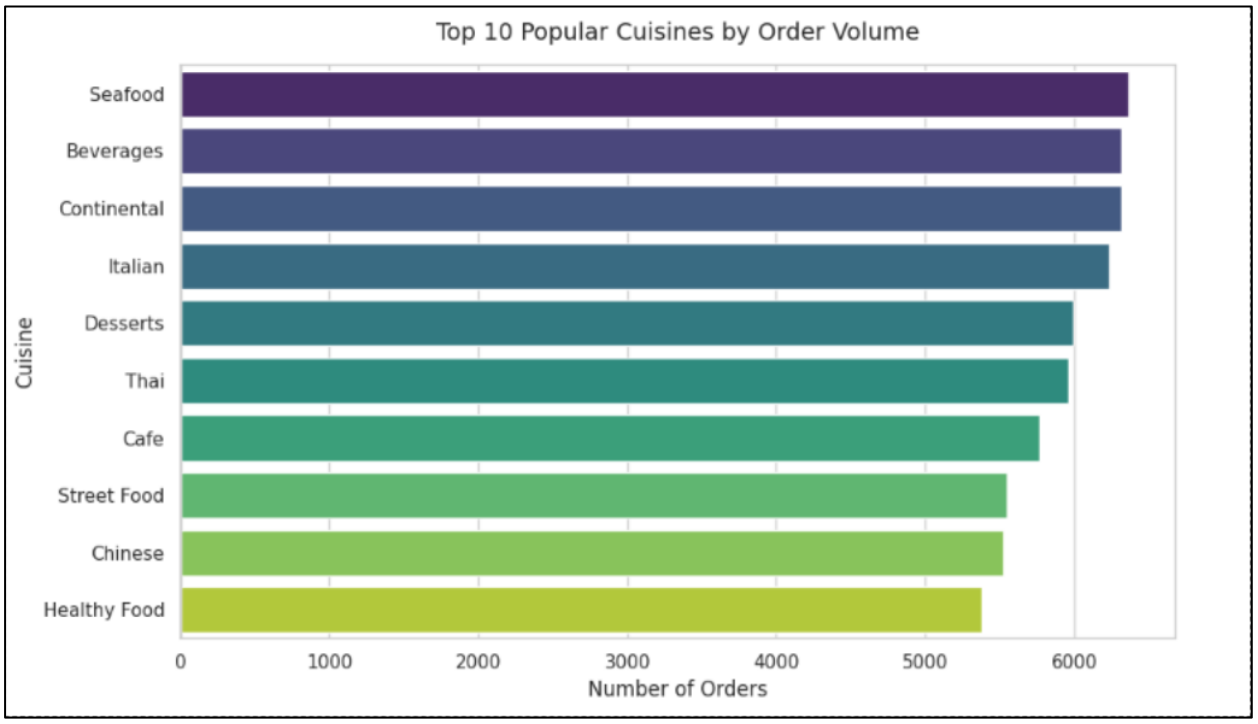


Figure 3.3: Top 10 Popular Cuisines by Order Volume

3.4.4 (Bar Chart) Order Distribution by Hour

A bar chart displays the number of orders for each recorded delivery hour. It helps identify the times of day when orders are most frequently recorded and understand daily order patterns. The delivery_hour column represents the hour of the recorded delivery time, not the duration taken to deliver an order.

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

sns.set_theme(style="whitegrid")

hourly_orders = (
```

```
    df['delivery_hour']
    .value_counts()
    .reindex(np.arange(24), fill_value=0)
    .rename_axis('Hour')
    .reset_index(name='Order Count')
)

plt.figure(figsize=(12, 6))

sns.barplot(
    data=hourly_orders,
    x='Hour',
    y='Order Count',
    hue='Hour',
    palette='magma',
    legend=False
)

plt.title("Order Distribution by Recorded Delivery Hour", fontsize=14,
pad=15)
plt.xlabel("Hour of Day", fontsize=12)
plt.ylabel("Number of Orders", fontsize=12)
plt.xticks(np.arange(24), [f"{h:02d}:00" for h in range(24)], rotation=45)
plt.tight_layout()
plt.show()
```

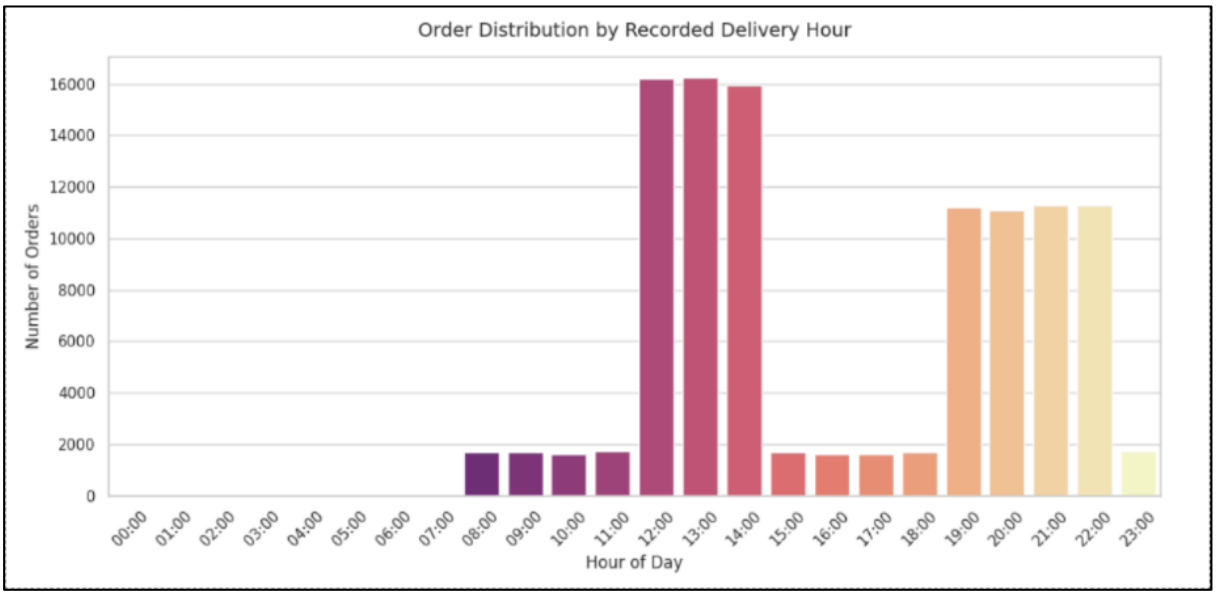


Figure 3.4: Order Distribution by Recorded Delivery Hour

3.4.5 (Histogram with KDE) Order Amount Distribution

A histogram with a Kernel Density Estimation (KDE) curve illustrates the distribution of food-order amounts. Mean and median lines help summarize customer spending and understand variations in transaction values.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

plt.figure(figsize=(10, 6))

sns.histplot(
    data=df,
    x='total_amount',
    bins=30,
    color='teal',
    edgecolor='black',
    kde=True
)

plt.axvline(
    df['total_amount'].mean(),
    color='red',
    linestyle='--',
    label=f"Mean: ₹{df['total_amount'].mean():.2f}"
)

plt.axvline(
    df['total_amount'].median(),
    color='blue',
    linestyle='-.',
    label=f"Median: ₹{df['total_amount'].median():.2f}"
)

plt.title("Distribution of Food Order Amounts", fontsize=14, pad=15)
plt.xlabel("Total Order Amount (₹)", fontsize=12)
plt.ylabel("Frequency of Orders", fontsize=12)
plt.legend()
plt.tight_layout()
```

```
plt.show()
```

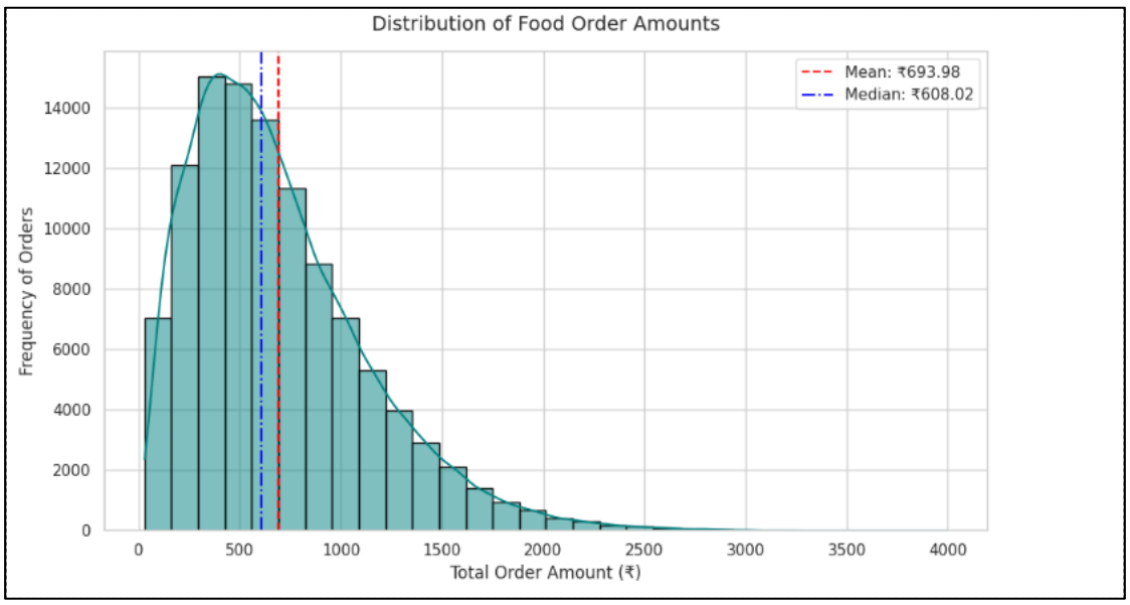


Figure 3.5: Distribution of Food Order Amounts

3.4.6 (Line Chart) Monthly Spending Trends

A line chart illustrates the monthly total value of food orders. It helps analyze changes in customer spending, compare transaction values across months, and identify trends in overall order revenue.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

df['order_date'] = pd.to_datetime(df['order_date'], errors='coerce')

monthly_amount = (
    df.dropna(subset=['order_date'])
      .set_index('order_date')['total_amount']
      .resample('ME')
      .sum()
      .reset_index()
)

monthly_amount.columns = ['Month', 'Total Order Amount']
monthly_amount['Month_Str'] = monthly_amount['Month'].dt.strftime('%b %Y')

plt.figure(figsize=(12, 6))

sns.lineplot(
    data=monthly_amount,
    x='Month_Str',
    y='Total Order Amount',
    marker='o',
    color='darkorange',
```

```
    linewidth=2.5
)

plt.title("Monthly Food Order Spending Trends", fontsize=14, pad=15)
plt.xlabel("Month", fontsize=12)
plt.ylabel("Total Order Amount (₹)", fontsize=12)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

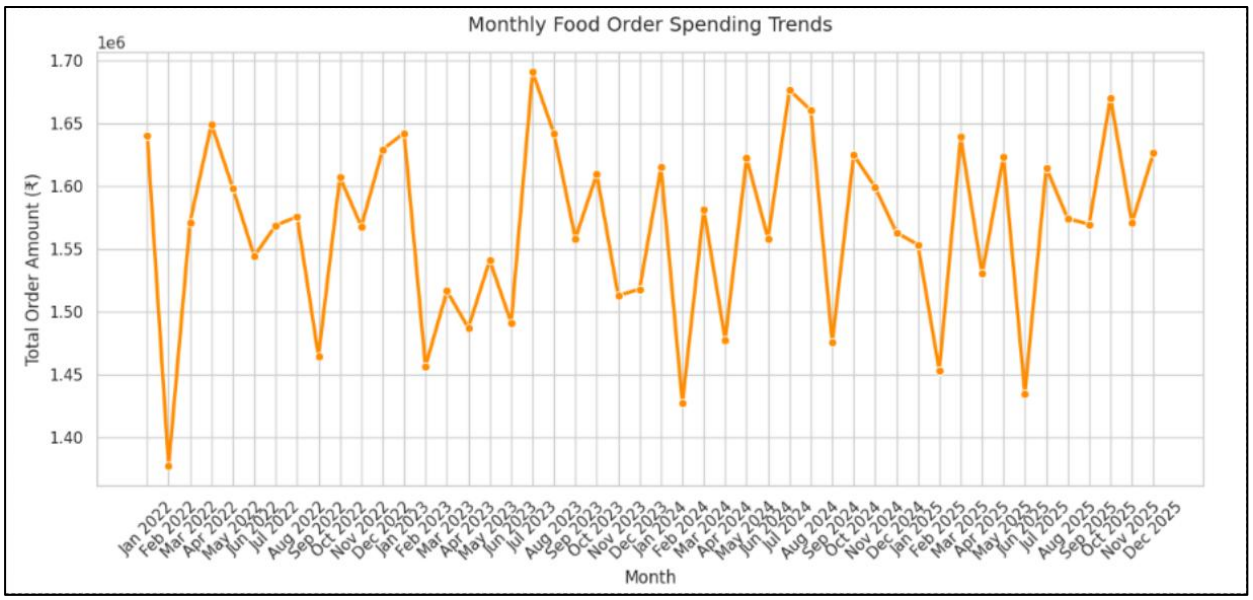


Figure 3.6: Monthly Food Order Spending Trends

3.5 Statistical Analysis

Statistical analysis can provide a more detailed summary of numerical patterns and relationships in the food delivery dataset. The choice of method depends on the variables being studied and whether the assumptions of the method are satisfied.

3.5.1 Statistical Methods Used

The following methods may be used where appropriate:

- **Descriptive statistics:** Summarize order amounts, customer ages, and ratings using measures such as mean, median, variance, and standard deviation.
- **Pearson correlation:** Examine linear relationships between numerical variables.
- **Spearman correlation:** Examine monotonic relationships when the assumptions for Pearson correlation are unsuitable or ordinal variables are involved.
- **Confidence intervals:** Estimate a plausible range for a population parameter, subject to the sampling assumptions.
- **Hypothesis tests:** Compare groups only when the research question, variable types, and assumptions justify the test.
- **Outlier analysis:** Identify observations outside IQR-based fences for further investigation.

A significance level of (\alpha = 0.05) may be used for hypothesis tests if these tests are actually performed. Statistical significance does not necessarily imply practical importance.

3.5.2 Probability / Distribution Analysis

The distribution of order amounts can be examined using a histogram, descriptive statistics, and measures of skewness. If a suitable probability distribution is used to estimate probabilities, its assumptions and fit should be checked first. Order amounts may be skewed, so a normal distribution should not be assumed without evidence.

The observed distribution of customer ratings can also be examined to understand how frequently different ratings occur.

3.5.3 Correlation

The correlation matrix provides information about the linear relationships between numerical variables. Correlation coefficients should be interpreted alongside sample size, data quality, and the characteristics of each variable.

The analysis should not claim that a variable causes higher spending or better ratings based on correlation alone.

3.5.4 Summary of Statistical Analysis

Table 3.9: Summary of Statistical Analysis

Analysis	Purpose	Result
Descriptive statistics	Summarize numerical variables	Insert actual results
Frequency analysis	Compare categorical counts	Insert actual results
IQR analysis	Identify potential numerical outliers	Insert actual results
Correlation analysis	Examine relationships between numerical variables	Insert actual results
Time-based analysis	Examine order counts over time	Insert actual results

3.5.5 Interpretation of Statistical Results

Table 3.9: Summary of hypothesis tests

Test	Statistic	p-value	Decision (α = 0.05)
Tuition vs no tuition (Welch t-test)	t = 3.60	< 0.001	Significant difference
Parental education (ANOVA)	F = 3.51	0.031	Significant difference
Male vs female (Welch t-test)	t = -0.18	0.857	No significant difference

Students who attend tuition score significantly higher (68.31 vs 65.05). Parental education has a statistically significant but small effect. There is no evidence of a gender difference in marks ($p = 0.86$). In summary, the strongest measurable influences on final marks are study hours, previous performance and attendance.

Chapter 4: Results and Findings

4.1 Key Observations

The analysis of the food delivery dataset identifies several important observations about ordering patterns, customer preferences, restaurant performance, and spending behaviour.

- **Order status:** The dataset contains 103,019 delivered orders and 5,365 cancelled orders. Delivered orders account for approximately 95.05% of the records, while cancelled orders account for approximately 4.95%.
- **Payment methods:** UPI is the most frequently used payment method, with 43,409 orders, followed by credit cards (27,197), debit cards (16,216), digital wallets (10,854), and cash on delivery (10,708).
- **Cloud kitchens:** The dataset contains 77,265 orders associated with non-cloud kitchens and 31,119 orders associated with cloud kitchens.
- **Customer spending:** The total_amount variable is used to examine order values, spending patterns, and unusually high or low amounts. The previously reported IQR analysis identified 2,582 potential outliers, representing approximately 2.38% of the dataset.
- **Customer ratings:** Ratings range from 2.5 to 5.0 in the observed dataset. Therefore, the rating distribution should be interpreted using the actual values rather than assuming that every possible rating between 1 and 5 is present.
- **Ordering patterns:** Order dates, cities, cuisines, and restaurant names can be analysed to identify temporal patterns, frequently ordered cuisines, and differences in ordering activity across locations.

These observations describe the available dataset and should be updated if the final notebook outputs differ.

4.2 Important Patterns Identified

- **Order completion pattern:** Most recorded orders are delivered successfully, while a smaller proportion are cancelled. Examining cancellation patterns can help identify areas for further investigation.
- **Payment preference pattern:** UPI accounts for the largest number of orders among the recorded payment methods, indicating its prominent use in this dataset.
- **Cloud kitchen pattern:** Both cloud kitchens and non-cloud kitchens contribute to the recorded orders. Their order counts can be compared to examine their participation in the food delivery dataset.

- **Customer spending variation:** Order amounts vary across transactions. IQR analysis helps identify unusually high or low values that may require investigation; these values should not automatically be treated as errors.
- **Cuisine and location patterns:** Grouping orders by cuisine, restaurant, and city can reveal differences in customer preferences and order volumes.
- **Rating pattern:** The observed rating range can be explored using frequency distributions and summary statistics to understand how customers rate their orders.

4.3 Summary of Results

Table 4.1: Summary of Key Findings from Food Delivery Data Analysis

Factor	Observed result	Evidence
Order status	Most orders are delivered	103,019 delivered; 5,365 cancelled
Payment method	UPI is the most frequently recorded method	43,409 UPI orders
Cloud kitchen	Non-cloud kitchens account for more recorded orders	77,265 non-cloud; 31,119 cloud
Order amount	Potential outliers were identified	2,582 IQR outliers reported previously
Customer ratings	Observed ratings range from 2.5 to 5.0	Dataset validation
Ordering patterns	Orders can be compared by city, cuisine, restaurant, and date	Grouping and exploratory data analysis

The results provide an overview of order completion, payment preferences, cloud-kitchen participation, customer ratings, and spending variation. Together, these findings demonstrate how data cleaning, exploratory data analysis, statistical summaries, and visualisation can be used to understand food delivery data.

Chapter 5: Conclusion and Future Scope

5.1 Conclusion

This project analysed food delivery data to understand ordering patterns, customer preferences, payment methods, order status, restaurant participation, customer ratings, and spending behaviour. Using the techniques covered in the Foundations of Data Science course, the dataset was examined through data loading, data cleaning, data validation, descriptive statistics, exploratory data analysis (EDA), visualisation, and statistical analysis.

The analysis showed that delivered orders form the majority of recorded transactions and UPI is the most frequently used payment method in the dataset. Comparisons between cloud kitchens and non-cloud kitchens provided insights into their respective order volumes. Customer ratings and order amounts were also examined to understand rating distributions and variations in spending. The use of charts, summary statistics, and outlier detection helped present the data in a structured and understandable form.

The project demonstrates how Python libraries such as Pandas, NumPy, Matplotlib, and Seaborn can be used to process and analyse food delivery data. The findings can help identify customer preferences and ordering trends that may be useful for further investigation by restaurants and food delivery platforms. However, the results describe the available dataset and should not automatically be generalised to all food delivery services.

5.2 Limitations

- Dataset representativeness: The dataset may not represent every food delivery platform, restaurant, city, or customer group. If the dataset is simulated, the findings must be validated using real-world data.
- Limited variables: The analysis is restricted to the attributes available in the dataset. Factors such as weather, traffic conditions, delivery distance, discounts, restaurant preparation time, and delivery-partner availability may also affect ordering and delivery outcomes.
- Association does not prove causation: Relationships between variables do not establish that one factor directly causes another.
- Delivery-time interpretation: The meaning and format of the delivery_time column must be confirmed before drawing conclusions about delivery duration or delivery efficiency.
- Outlier interpretation: IQR-based outlier detection identifies unusual values but does not establish that those values are incorrect. Each potential outlier requires appropriate

interpretation.

- Limited predictive analysis: The project primarily focuses on data analysis and visualisation. Predictive modelling and deployment of a real-time analytical system are outside the current scope unless implemented separately.

5.3 Future Scope

- **Use real-world datasets:** Extend the analysis using verified data from food delivery services across multiple cities and time periods to assess whether the observed patterns are consistent.
- **Develop predictive models:** Apply suitable machine-learning algorithms to predict order cancellation, customer ratings, or order demand, provided that the dataset contains appropriate input variables and sufficient reliable observations.
- **Improve delivery analysis:** If accurate timestamps or delivery-duration data become available, investigate delivery delays and factors associated with longer delivery times.
- **Perform advanced statistical analysis:** Use suitable hypothesis tests, confidence intervals, regression methods, and other statistical techniques to investigate differences and relationships between variables.
- **Build an interactive dashboard:** Develop a dashboard using Power BI or Streamlit to display order volumes, popular cuisines, payment preferences, cancellation rates, ratings, and spending patterns.
- **Analyse customer behaviour over time:** Use additional historical data to examine seasonal trends, changes in cuisine preferences, repeat ordering, and customer spending over time.
- **Improve data quality:** Add systematic validation rules for dates, ratings, order amounts, categories, and identifiers to make future analyses more reliable.

In conclusion, the Food Delivery Data Analysis project demonstrates the practical application of data science techniques to a real-world problem domain. The project provides a foundation for more advanced statistical analysis, predictive modelling, and interactive reporting as more detailed and reliable data becomes available.

REFERENCES

[1] W. McKinney, *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

[2] J. VanderPlas, *Python Data Science Handbook: Essential Tools for Working with Data*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

[3] P. Bruce, A. Bruce, and P. Gedeck, *Practical Statistics for Data Scientists: 50+ Essential Concepts Using R and Python*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.

[4] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[5] P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[6] M. L. Waskom, "seaborn: Statistical Data Visualization," *Journal of Open Source Software*, vol. 6, no. 60, Art. no. 3021, 2021.

[7] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61.

[8] Pandas Development Team, "pandas Documentation." [Online]. Available: https://pandas.pydata.org/docs/

[9] NumPy Developers, "NumPy Documentation." [Online]. Available: https://numpy.org/doc/

[10] Matplotlib Development Team, "Matplotlib Documentation." [Online]. Available: https://matplotlib.org/stable/

[11] SciPy Community, “SciPy Documentation.” [Online]. Available: https://docs.scipy.org/doc/scipy/

[12] M. L. Waskom, “seaborn Documentation.” [Online]. Available: https://seaborn.pydata.org/

[13] Project Jupyter, “Jupyter Notebook Documentation.” [Online]. Available: https://jupyter-notebook.readthedocs.io/

[14] Google, “Google Colaboratory.” [Online]. Available: https://colab.research.google.com/

[15] Food Delivery Data Analysis Project, *Food_Delivery_Data_Analysis.ipynb*, project notebook, GitHub repository: https://github.com/pranav662/Food_Delivery_Data_Analysis_FDS_Project

APPENDIX – PYTHON CODE

1. Import Libraries and Load Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import io

from google.colab import files

sns.set_theme(style="whitegrid")
plt.rcParams["figure.dpi"] = 100
pd.set_option("display.max_columns", None)

# Configure loading method: "UPLOAD" or "DRIVE"
LOAD_METHOD = "DRIVE"

if LOAD_METHOD == "UPLOAD":
    print("Please upload Swiggy_Data.csv:")
    uploaded = files.upload()

    if "Swiggy_Data.csv" not in uploaded:
        raise FileNotFoundError(
            "Please upload the file named Swiggy_Data.csv."
        )

    df = pd.read_csv(io.BytesIO(uploaded["Swiggy_Data.csv"]))

else:
    from google.colab import drive

    drive.mount("/content/drive", force_remount=True)

    drive_path = (
        "/content/drive/MyDrive/"
        "Food_Delivery_Data_Analysis/Swiggy_Data.csv"
    )

    if not os.path.exists(drive_path):
```

```
        raise FileNotFoundError(
            f"Dataset not found at: {drive_path}"
        )

    df = pd.read_csv(drive_path)

if df is None or df.empty:
    raise ValueError("The loaded dataset is empty or corrupted.")

print("Dataset shape:", df.shape)
print("Columns:", df.columns.tolist())
print("\nData types:")
print(df.dtypes)

display(df.head())
```

2. Initial Structural Inspection and Profiling

```
print("Dataset dimensions:", df.shape)

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nDataset information:")
df.info()

print("\nStatistical summary:")
display(df.describe(include="all").T)

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())
```

3. Data Preprocessing and Cleaning

```
# Check duplicate rows and duplicate order IDs
duplicate_rows_count = df.duplicated().sum()
duplicate_order_ids = df["order_id"].duplicated().sum()
```

```
print("Exact duplicate rows:", duplicate_rows_count)
print("Duplicate order IDs:", duplicate_order_ids)

# Remove exact duplicate rows if present
if duplicate_rows_count > 0:
    df = df.drop_duplicates().copy()

# Convert order_date to datetime
df["order_date"] = pd.to_datetime(
    df["order_date"].astype(str).str.strip(),
    format="%Y-%m-%d",
    errors="coerce"
)

# Convert numeric columns
for col in ["total_amount", "rating", "age"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# Standardize text columns
text_cols = [
    "order_status",
    "payment_method",
    "restaurant_name",
    "city",
    "cuisine",
    "gender",
    "marital_status",
    "occupation"
]

for col in text_cols:
    df[col] = df[col].astype("string").str.strip()
    df[col] = df[col].replace("", pd.NA)

# Parse delivery time as a time of day
delivery_str = df["delivery_time"].astype(str).str.strip()

parsed_delivery_time = pd.to_datetime(
    delivery_str,
    format="%H:%M:%S",
    errors="coerce"
)
```

```
print(
    "Successfully parsed delivery times:",
    parsed_delivery_time.notna().sum()
)

print(
    "Missing or malformed delivery times:",
    parsed_delivery_time.isna().sum()
)

# Extract delivery hour
df["delivery_hour"] = parsed_delivery_time.dt.hour

print("Shape after preprocessing:", df.shape)

print("\nMissing values after conversion:")
print(df[["order_date", "total_amount", "rating", "age"]].isnull().sum())

print("\nObserved rating range:")
print(df["rating"].min(), "to", df["rating"].max())

print("\nObserved age range:")
print(df["age"].min(), "to", df["age"].max())

print("\nInvalid rating count:")
print((~df["rating"].between(1, 5)).sum())

print("\nInvalid age count:")
print((~df["age"].between(18, 100)).sum())

print("\nDuplicate order IDs:")
print(df["order_id"].duplicated().sum())
```

4. Data Validation and Quality Diagnostics

```
total_missing_values = df.isnull().sum().sum()
missing_by_column = df.isnull().sum().to_dict()

missing_order_id = df["order_id"].isna().sum()
missing_user_id = df["user_id"].isna().sum()
missing_restaurant_id = df["restaurant_id"].isna().sum()

duplicate_complete_rows = df.duplicated().sum()
```

```
duplicate_order_ids_count = df["order_id"].duplicated().sum()

invalid_dates = df["order_date"].isna().sum()
malformed_times = parsed_delivery_time.isna().sum()

invalid_ratings_count = (
    (df["rating"] < 1)
    | (df["rating"] > 5)
    | df["rating"].isna()
).sum()

invalid_ages_count = (
    (df["age"] < 18)
    | (df["age"] > 100)
    | df["age"].isna()
).sum()

invalid_amounts_count = (
    df["total_amount"].isna()
    | (df["total_amount"] <= 0)
).sum()

allowed_statuses = ["Delivered", "Cancelled"]

invalid_statuses_count = (
    ~df["order_status"].isin(allowed_statuses)
).sum()

allowed_payments = [
    "UPI",
    "Credit Card",
    "Debit Card",
    "Wallet",
    "Cash on Delivery"
]

invalid_payments_count = (
    ~df["payment_method"].isin(allowed_payments)
).sum()

validation_tests = [
    {
        "Validation Parameter": "Missing order IDs",
```

```
        "Affected Records": missing_order_id
    },
    {
        "Validation Parameter": "Missing user/restaurant IDs",
        "Affected Records": missing_user_id + missing_restaurant_id
    },
    {
        "Validation Parameter": "Duplicate complete rows",
        "Affected Records": duplicate_complete_rows
    },
    {
        "Validation Parameter": "Duplicate order IDs",
        "Affected Records": duplicate_order_ids_count
    },
    {
        "Validation Parameter": "Invalid order dates",
        "Affected Records": invalid_dates
    },
    {
        "Validation Parameter": "Malformed delivery times",
        "Affected Records": malformed_times
    },
    {
        "Validation Parameter": "Invalid ratings",
        "Affected Records": invalid_ratings_count
    },
    {
        "Validation Parameter": "Invalid ages",
        "Affected Records": invalid_ages_count
    },
    {
        "Validation Parameter": "Invalid order amounts",
        "Affected Records": invalid_amounts_count
    },
    {
        "Validation Parameter": "Invalid order statuses",
        "Affected Records": invalid_statuses_count
    },
    {
        "Validation Parameter": "Invalid payment methods",
        "Affected Records": invalid_payments_count
    }
]
```

```
diagnostics_summary_df = pd.DataFrame(validation_tests)

display(diagnostics_summary_df)

print("\nMissing-value profile:")
for col, val in missing_by_column.items():
    percentage = (val / len(df)) * 100
    print(
        f"{col:<20} | Missing: {val:<6} | "
        f"Percentage: {percentage:.2f}%"
    )
```

5. Outlier Auditing Using the IQR Method

```
iqr_data = []
target_cols = ["total_amount", "age"]

for col in target_cols:
    if col in df.columns and pd.api.types.is_numeric_dtype(df[col]):
        clean_series = df[col].dropna()

        if clean_series.empty:
            continue

        q1 = clean_series.quantile(0.25)
        q3 = clean_series.quantile(0.75)
        iqr = q3 - q1

        lower_bound = q1 - 1.5 * iqr
        upper_bound = q3 + 1.5 * iqr

        outliers = clean_series[
            (clean_series < lower_bound)
            | (clean_series > upper_bound)
        ]

        outlier_count = len(outliers)
        outlier_percent = outlier_count / len(clean_series) * 100

        iqr_data.append({
            "Column": col,
            "Q1": round(q1, 2),
```

```
            "Q3": round(q3, 2),
            "IQR": round(iqr, 2),
            "Lower Bound": round(lower_bound, 2),
            "Upper Bound": round(upper_bound, 2),
            "Outlier Count": outlier_count,
            "Outlier Percentage": f"{outlier_percent:.2f}%"
        })

iqr_summary_df = pd.DataFrame(iqr_data)
display(iqr_summary_df)

# Inspect high-value order transactions
amount_bounds = iqr_summary_df.loc[
    iqr_summary_df["Column"] == "total_amount",
    "Upper Bound"
]

if not amount_bounds.empty:
    upper_amount_bound = amount_bounds.iloc[0]

    high_value_outliers = df[
        df["total_amount"] > upper_amount_bound
    ]

    display(
        high_value_outliers[
            [
                "order_id",
                "restaurant_name",
                "cuisine",
                "total_amount",
                "rating"
            ]
        ].head(5)
    )

# Box plot for order amounts
plt.figure(figsize=(12, 5))

sns.boxplot(
    data=df,
    x="total_amount",
    color="skyblue",
```

```
    flierprops={
        "markerfacecolor": "red",
        "marker": "D",
        "markersize": 4,
        "alpha": 0.5
    }
)

plt.title("Outlier Analysis of Order Amounts")
plt.xlabel("Total Order Amount (₹)")
plt.tight_layout()
plt.show()
```

6. Exploratory Data Analysis (EDA)

```
# Order status analysis
status_summary = df["order_status"].value_counts().to_frame("Orders")

status_summary["Percentage"] = (
    status_summary["Orders"] / len(df) * 100
).round(2)

print("Order Status Summary")
display(status_summary)

# Payment method analysis
payment_summary = df["payment_method"].value_counts().to_frame("Orders")

payment_summary["Percentage"] = (
    payment_summary["Orders"] / len(df) * 100
).round(2)

print("\nPayment Method Summary")
display(payment_summary)

# Top 10 cuisines
cuisine_summary = (
    df["cuisine"].value_counts().head(10).to_frame("Orders")
)

print("\nTop 10 Cuisines")
display(cuisine_summary)
```

```
# Top 10 cities
city_summary = (
    df["city"].value_counts().head(10).to_frame("Orders")
)

print("\nTop 10 Cities")
display(city_summary)

# Order amount statistics
display(df["total_amount"].describe().to_frame("Total Amount"))

# Rating statistics
display(df["rating"].describe().to_frame("Rating"))

# Delivery completion hour distribution
hour_summary = (
    df["delivery_hour"]
    .value_counts()
    .sort_index()
    .to_frame("Orders Completion Count")
)

hour_summary["Percentage"] = (
    hour_summary["Orders Completion Count"] / len(df) * 100
).round(2)

display(hour_summary)
```

7. Descriptive Statistical Analysis

```
numeric_cols = ["total_amount", "rating", "age"]
stats_dict = {}

for col in numeric_cols:
    if (
        col in df.columns
        and pd.api.types.is_numeric_dtype(df[col])
    ):
        col_data = df[col].dropna()

        if not col_data.empty:
            mode_series = col_data.mode()
            mode_val = (
```

```
                mode_series.iloc[0]
                if not mode_series.empty
                else np.nan
            )

            unit = ""
            if col == "total_amount":
                unit = " (₹)"
            elif col == "age":
                unit = " (Years)"
            elif col == "rating":
                unit = " (Permitted: 1–5; observed: 2.5–5.0)"

            stats_dict[col + unit] = {
                "Mean": round(col_data.mean(), 2),
                "Median": round(col_data.median(), 2),
                "Mode": round(mode_val, 2),
                "Variance (Sample)": round(col_data.var(ddof=1), 2),
                "Std Deviation (Sample)": round(col_data.std(ddof=1), 2),
                "Minimum": round(col_data.min(), 2),
                "Maximum": round(col_data.max(), 2),
                "Range": round(
                    col_data.max() - col_data.min(), 2
                )
            }

descriptive_stats_df = pd.DataFrame(stats_dict).T

display(descriptive_stats_df)
```

8. Pearson Correlation Analysis

```
correlation_cols = ["total_amount", "rating", "age"]

corr_data = df[correlation_cols].dropna()

constant_cols = [
    col for col in corr_data.columns
    if corr_data[col].nunique() <= 1
]

corr_cols_filtered = [
    col for col in correlation_cols
```

```
    if col not in constant_cols
]

correlation_matrix = corr_data[
    corr_cols_filtered
].corr(method="pearson")

print("Correlation Matrix")
display(correlation_matrix.round(3))

if not correlation_matrix.empty:
    plt.figure(figsize=(8, 6))

    sns.heatmap(
        correlation_matrix,
        annot=True,
        cmap="coolwarm",
        fmt=".3f",
        vmin=-1,
        vmax=1,
        center=0,
        square=True,
        linewidths=0.5,
        cbar_kws={"shrink": 0.8}
    )

    plt.title(
        "Correlation Heatmap: Order Amount, Rating, and Age"
    )
    plt.tight_layout()
    plt.show()
```

9. Data Visualization

9.1 Top 10 Popular Cuisines

```
plt.figure(figsize=(10, 6))

top_cuisines = (
    df["cuisine"]
    .value_counts()
    .head(10)
    .reset_index()
```

```
)

top_cuisines.columns = ["Cuisine", "Order Count"]

sns.barplot(
    x="Order Count",
    y="Cuisine",
    data=top_cuisines,
    hue="Cuisine",
    palette="viridis",
    legend=False
)

plt.title("Top 10 Popular Cuisines by Order Volume")
plt.xlabel("Number of Orders")
plt.ylabel("Cuisine")
plt.tight_layout()
plt.show()
```

9.2 Order Amount Distribution and Monthly Trend

```
fig, axes = plt.subplots(1, 2, figsize=(18, 6))

# Order amount distribution
sns.histplot(
    data=df,
    x="total_amount",
    bins=30,
    color="teal",
    edgecolor="black",
    kde=True,
    ax=axes[0]
)

axes[0].set_title("Distribution of Order Amounts")
axes[0].set_xlabel("Total Order Amount (₹)")
axes[0].set_ylabel("Transaction Frequency")

axes[0].axvline(
    df["total_amount"].mean(),
    color="red",
    linestyle="--",
    label=f"Mean: ₹{df['total_amount'].mean():.2f}"
```

```
)

axes[0].axvline(
    df["total_amount"].median(),
    color="blue",
    linestyle="-.",
    label=f"Median: ₹{df['total_amount'].median():.2f}"
)

axes[0].legend()

# Monthly order trend
monthly_trend = (
    df.dropna(subset=["order_date"])
    .set_index("order_date")
    .resample("ME")
    .size()
    .reset_index()
)

monthly_trend.columns = ["Month", "Order Count"]
monthly_trend["Month_Str"] = monthly_trend["Month"].dt.strftime("%b %Y")

sns.lineplot(
    data=monthly_trend,
    x="Month_Str",
    y="Order Count",
    marker="o",
    color="darkorange",
    linewidth=2.5,
    ax=axes[1]
)

axes[1].set_title("Monthly Order Volumes")
axes[1].set_xlabel("Month")
axes[1].set_ylabel("Number of Orders")
axes[1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```

9.3 Payment Method Distribution and Delivery Hour

```
fig, axes = plt.subplots(1, 2, figsize=(18, 8))

# Payment method pie chart
payment_counts = df["payment_method"].value_counts()

axes[0].pie(
    payment_counts,
    labels=payment_counts.index,
    autopct="%1.2f%%",
    startangle=140,
    colors=sns.color_palette("pastel", n_colors=len(payment_counts)),
    wedgeprops={"edgecolor": "white", "linewidth": 1}
)

axes[0].set_title("Payment Method Distribution")

# Orders by recorded delivery completion hour
hourly_counts = (
    df["delivery_hour"]
    .value_counts()
    .reindex(np.arange(24), fill_value=0)
    .rename_axis("Hour of Day")
    .reset_index(name="Total Orders")
)

sns.barplot(
    data=hourly_counts,
    x="Hour of Day",
    y="Total Orders",
    hue="Hour of Day",
    palette="magma",
    legend=False,
    ax=axes[1]
)

axes[1].set_title("Orders by Recorded Completion Hour")
axes[1].set_xlabel("Hour of Day")
axes[1].set_ylabel("Number of Orders")
axes[1].set_xticks(np.arange(24))
axes[1].set_xticklabels(
    [f"{hour:02d}:00" for hour in range(24)],
    rotation=45
)
```

```
plt.tight_layout()
plt.show()
```

9.4 Restaurant Rating Distribution

```
plt.figure(figsize=(10, 5))

sns.boxplot(
    data=df,
    x="rating",
    color="lightgreen",
    flierprops={
        "markerfacecolor": "red",
        "marker": "D",
        "markersize": 5
    }
)

plt.title("Distribution of Restaurant Ratings")
plt.xlabel("Rating (Permitted: 1–5; observed: 2.5–5.0)")
plt.tight_layout()
plt.show()
```

9.5 Order Amount vs. Restaurant Rating

```
plt.figure(figsize=(10, 6))

sns.scatterplot(
    data=df,
    x="total_amount",
    y="rating",
    alpha=0.3,
    color="purple"
)

sns.regplot(
    data=df,
    x="total_amount",
    y="rating",
    scatter=False,
    color="red",
    line_kws={"linestyle": "--", "linewidth": 1.5}
```

```
)

plt.title("Order Amount vs. Restaurant Rating")
plt.xlabel("Total Order Amount (₹)")
plt.ylabel("Restaurant Rating")
plt.tight_layout()
plt.show()

# Pearson correlation
corr_amt_rating = df["total_amount"].corr(df["rating"])

print(
    "Pearson correlation between order amount and rating:",
    round(corr_amt_rating, 5)
)
```

10. Quantitative Validation of Findings

```
total_rows = len(df)
unique_cols_count = df.shape[1]

# Top cuisine and city
top_cuisine_name = df["cuisine"].value_counts().index[0]
top_cuisine_count = df["cuisine"].value_counts().iloc[0]

top_city_name = df["city"].value_counts().index[0]
top_city_count = df["city"].value_counts().iloc[0]

# Order amount statistics
mean_amt = df["total_amount"].mean()
median_amt = df["total_amount"].median()
std_amt = df["total_amount"].std()

# UPI payment statistics
upi_count = (df["payment_method"] == "UPI").sum()
upi_percentage = upi_count / total_rows * 100

# Rating and correlation
median_rating = df["rating"].median()
corr_amt_rating = df["total_amount"].corr(df["rating"])

# Spending bracket
spending_bracket_count = (
```

```
    df["total_amount"].between(100, 1000)
).sum()

spending_bracket_pct = spending_bracket_count / total_rows * 100

print("--- QUANTITATIVE VERIFICATION REPORT ---")
print("Total cleaned rows:", total_rows)
print("Columns:", unique_cols_count)
print(f"Top cuisine: {top_cuisine_name} ({top_cuisine_count} orders)")
print(f"Top city: {top_city_name} ({top_city_count} orders)")
print(f"Mean order amount: ₹{mean_amt:.2f}")
print(f"Median order amount: ₹{median_amt:.2f}")
print(f"Standard deviation: ₹{std_amt:.2f}")
print(f"UPI orders: {upi_count} ({upi_percentage:.2f}%)")
print(f"Median rating: {median_rating:.2f}")
print(f"Amount-rating correlation: {corr_amt_rating:.5f}")
print(
    f"Orders between ₹100 and ₹1,000: "
    f"{spending_bracket_count} ({spending_bracket_pct:.2f}%)"
)
```

11. Export Cleaned Data and Statistical Results

```
# Export cleaned dataset
df.to_csv("Swiggy_Data_Cleaned.csv", index=False)

# Export descriptive statistics
descriptive_stats_export = (
    descriptive_stats_df
    .reset_index()
    .rename(columns={"index": "Statistical Metric"})
)

descriptive_stats_export.to_csv(
    "Swiggy_Statistical_Summary.csv",
    index=False
)

# Export correlation matrix
correlation_matrix.to_csv(
    "Swiggy_Correlation_Matrix.csv",
    index=True
)
```

```
print("Export completed:")
print("1. Swiggy_Data_Cleaned.csv")
print("2. Swiggy_Statistical_Summary.csv")
print("3. Swiggy_Correlation_Matrix.csv")

# Optional: download files directly in Google Colab
# files.download("Swiggy_Data_Cleaned.csv")
# files.download("Swiggy_Statistical_Summary.csv")
# files.download("Swiggy_Correlation_Matrix.csv")
```